
AI-102 Natural Language Processing (NLP)

Dr. Jimmy Ardiansyah



EMERGING &
COLLABORATIVE STUDIES





Machine Translation

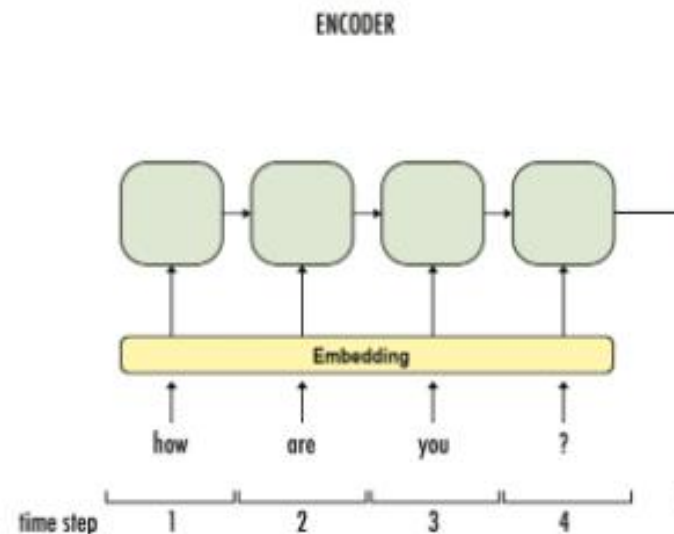
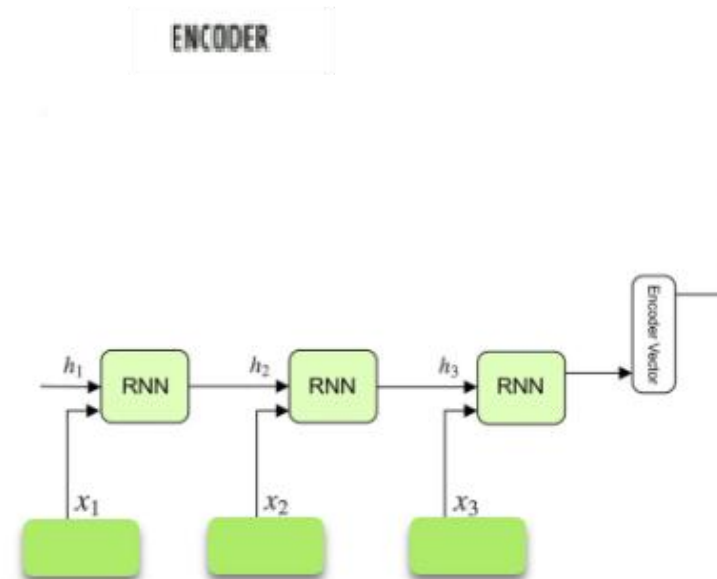
What Are Seq2Seq Models?

Seq2Seq (Sequence-to-Sequence) models are machine learning models designed to map an input sequence to an output sequence, which are useful for tasks where the input and output lengths differ, like machine translation or text summarization. They consist of two main parts: an **encoder** that processes the input into a context vector, and a **decoder** that uses the context vector to generate the output sequence

What Are Seq2Seq Models?

Seq2Seq (Sequence-to-Sequence) models are machine learning models designed to map an input sequence to an output sequence, which are useful for tasks where the input and output lengths differ, like machine translation or text summarization. They consist of two main parts: an **encoder** that processes the input into a context vector, and a **decoder** that uses the context vector to generate the output sequence

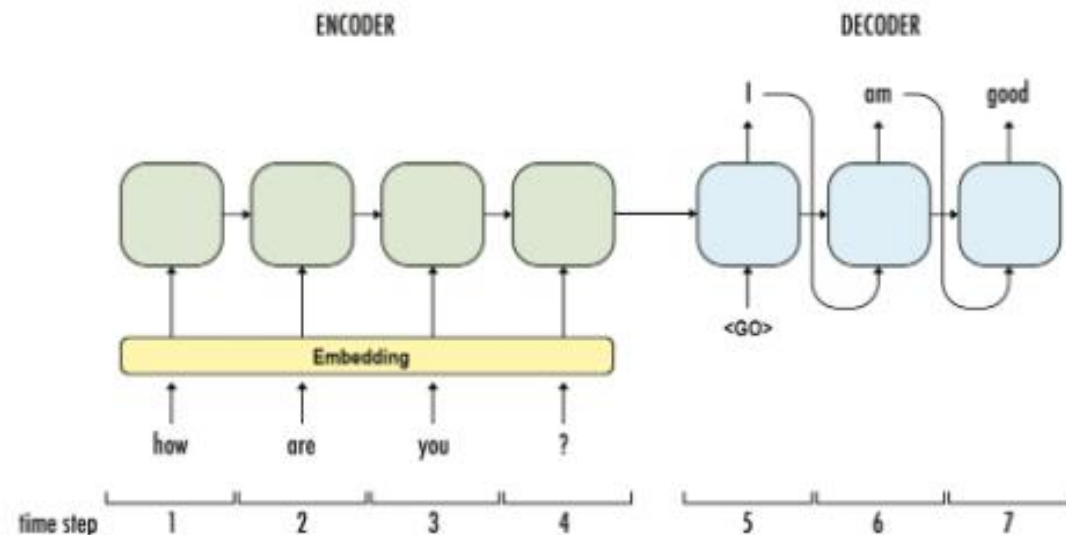
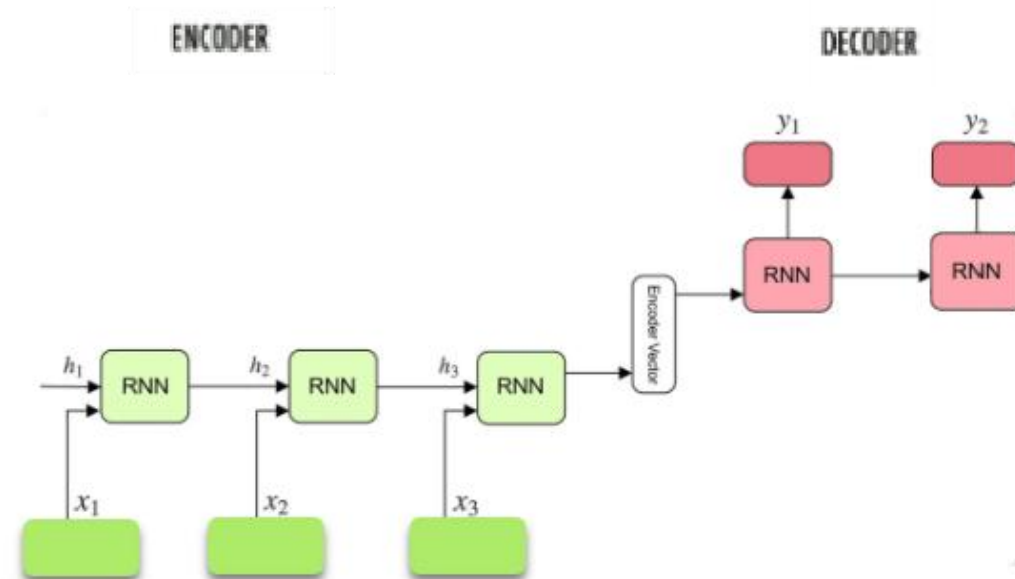
- Encoder
 - Reads the **input sequence** token by token
 - Uses RNN / LSTM / GRU cells
 - Produces a **fixed-size context vector** (hidden state) that summarizes the whole input



What Are Seq2Seq Models?

Seq2Seq (Sequence-to-Sequence) models are machine learning models designed to map an input sequence to an output sequence, which are useful for tasks where the input and output lengths differ, like machine translation or text summarization. They consist of two main parts: an **encoder** that processes the input into a context vector, and a **decoder** that uses the context vector to generate the output sequence

- Encoder
 - Reads the **input sequence** token by token
 - Uses **RNN / LSTM / GRU** cells
 - Produces a **fixed-size context vector** (hidden state) that summarizes the whole input
- Decoder
 - Takes the **context vector** and a **start token**
 - Generates the **output sequence** one token at a time
 - Each step uses previous token + hidden state to predict the next token
 - Stops when it emits an **end token** or hits max length



Advantages and Limitation Seq2Seq Models



- **Flexible:** handle variable-length input and output sequences
- **Context-aware:** encoder's context vector captures dependencies in the input
- **End-to-end training:** whole system trained jointly from examples



Advantages and Limitation Seq2Seq Models



- **Flexible:** handle variable-length input and output sequences
- **Context-aware:** encoder's context vector captures dependencies in the input
- **End-to-end training:** whole system trained jointly from examples



- **Fixed-length context bottleneck:** hard to encode all info for **long sequences** into one vector
- **Training is heavy:** needs **lots of data** and GPU/TPU resources
- **Exposure bias:** trained with **true previous tokens**, but at test time uses its **own predictions**
- **Limited interpretability:** hard to see *why* a specific output was generated

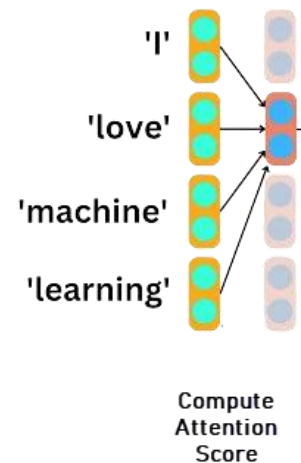
What Attention Mechanisms?

An attention mechanism in machine translation allows the model to focus on relevant parts of the source sentence when generating each word of the translated sentence, improving translation quality and coherence. It addresses the bottleneck of older models by enabling the decoder to selectively "look back" at all encoder hidden states, weighting them to create a dynamic context vector for each output step

What Attention Mechanisms?

An attention mechanism in machine translation allows the model to focus on relevant parts of the source sentence when generating each word of the translated sentence, improving translation quality and coherence. It addresses the bottleneck of older models by enabling the decoder to selectively "look back" at all encoder hidden states, weighting them to create a dynamic context vector for each output step

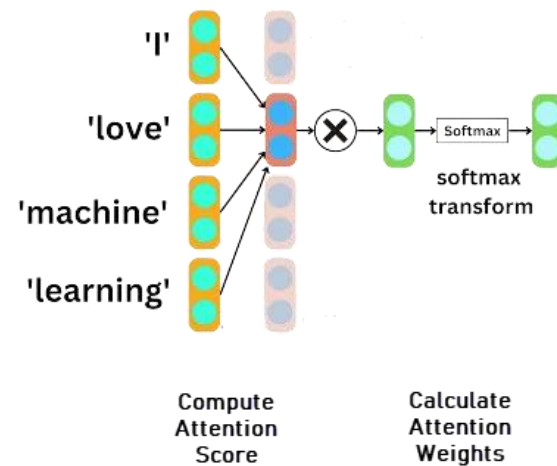
- **Compute Attention Scores**
 - Produces a **fixed-size context vector** (hidden
 - Methods: Dot-Product Attention: Takes the dot product of the states. (Simple, efficient) and Additive Attention (Bahdanau): Concatenates states and passes them through a neural network



What Attention Mechanisms?

An attention mechanism in machine translation allows the model to focus on relevant parts of the source sentence when generating each word of the translated sentence, improving translation quality and coherence. It addresses the bottleneck of older models by enabling the decoder to selectively "look back" at all encoder hidden states, weighting them to create a dynamic context vector for each output step

- **Compute Attention Scores**
 - Produces a **fixed-size context vector** (hidden)
 - Methods: Dot-Product Attention: Takes the dot product of the states. (Simple, efficient) and Additive Attention (Bahdanau): Concatenates states and passes them through a neural network
- **Calculate Attention Weights**
 - Apply the Softmax function to the attention scores and convert the scores into a probability distribution (weights sum to 1)

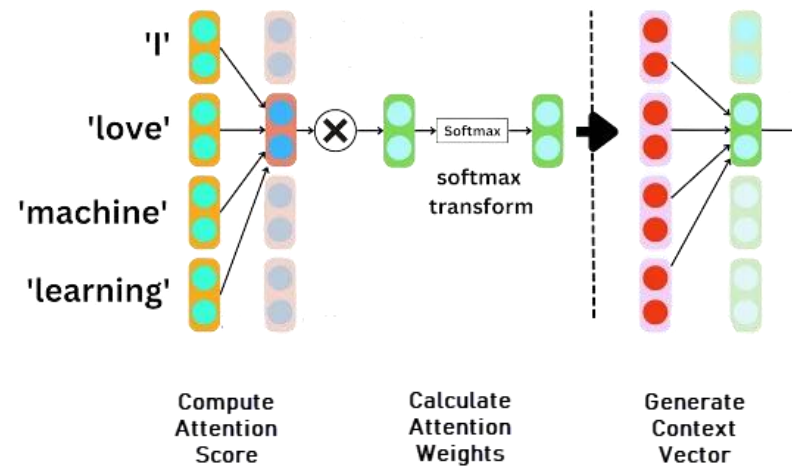


What Attention Mechanisms?

An attention mechanism in machine translation allows the model to focus on relevant parts of the source sentence when generating each word of the translated sentence, improving translation quality and coherence. It addresses the bottleneck of older models by enabling the decoder to selectively "look back" at all encoder hidden states, weighting them to create a dynamic context vector for each output step

- **Compute Attention Scores**
 - Produces a **fixed-size context vector** (hidden)
 - Methods: Dot-Product Attention: Takes the dot product of the states. (Simple, efficient) and Additive Attention (Bahdanau): Concatenates states and passes them through a neural network
- **Calculate Attention Weights**
 - Apply the Softmax function to the attention scores and convert the scores into a probability distribution (weights sum to 1)

- **Generate Context Vector**
 - Compute the weighted sum of the encoder hidden states, using the attention weights

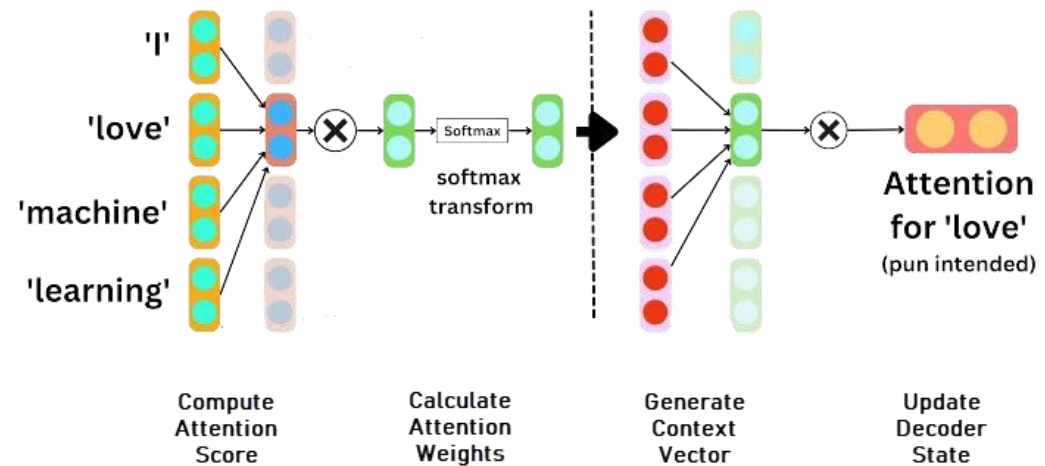


What Attention Mechanisms?

An attention mechanism in machine translation allows the model to focus on relevant parts of the source sentence when generating each word of the translated sentence, improving translation quality and coherence. It addresses the bottleneck of older models by enabling the decoder to selectively "look back" at all encoder hidden states, weighting them to create a dynamic context vector for each output step

- **Compute Attention Scores**
 - Produces a **fixed-size context vector** (hidden)
 - Methods: Dot-Product Attention: Takes the dot product of the states. (Simple, efficient) and Additive Attention (Bahdanau): Concatenates states and passes them through a neural network
- **Calculate Attention Weights**
 - Apply the Softmax function to the attention scores and convert the scores into a probability distribution (weights sum to 1)

- **Generate Context Vector**
 - Compute the weighted sum of the encoder hidden states, using the attention weights
- **Update Decoder State**
 - Combine the new context vector with the current decoder hidden state.



Advantages and Limitation Attention Mechanisms



- **Improved Performance:** Leads to more accurate and contextually relevant outputs (e.g., better word alignment in translation)
- **Handling Long Sequences:** Directly accesses the entire input sequence at every step, preventing information loss over time
- **Flexibility:** Easily integrated into various architectures like RNNs, LSTMs, and GRUs



Advantages and Limitation Attention Mechanisms



- **Improved Performance:** Leads to more accurate and contextually relevant outputs (e.g., better word alignment in translation)
- **Handling Long Sequences:** Directly accesses the entire input sequence at every step, preventing information loss over time
- **Flexibility:** Easily integrated into various architectures like RNNs, LSTMs, and GRUs



- **Complexity:** Increases model complexity, requiring more computational resources (memory and processing power).
- **Training Time:** Involves extra computations at every decoding step, leading to **longer training times**

What Transformer Models?

Transformer models are a type of neural network architecture that use a **self-attention mechanism** to perform machine translation and other language tasks. They revolutionized the field by enabling **parallel processing** of entire text sequences, which makes them faster and more efficient than previous models like Recurrent Neural Networks (RNNs)

What Transformer Models?

Transformer models are a type of neural network architecture that use a **self-attention mechanism** to perform machine translation and other language tasks. They revolutionized the field by enabling **parallel processing** of entire text sequences, which makes them faster and more efficient than previous models like Recurrent Neural Networks (RNNs)

The Encoder

- Function: Processes the entire input sequence (e.g., the source sentence).
- Output: Generates a set of context-rich representations that capture intricate patterns and relationships within the input data

What Transformer Models?

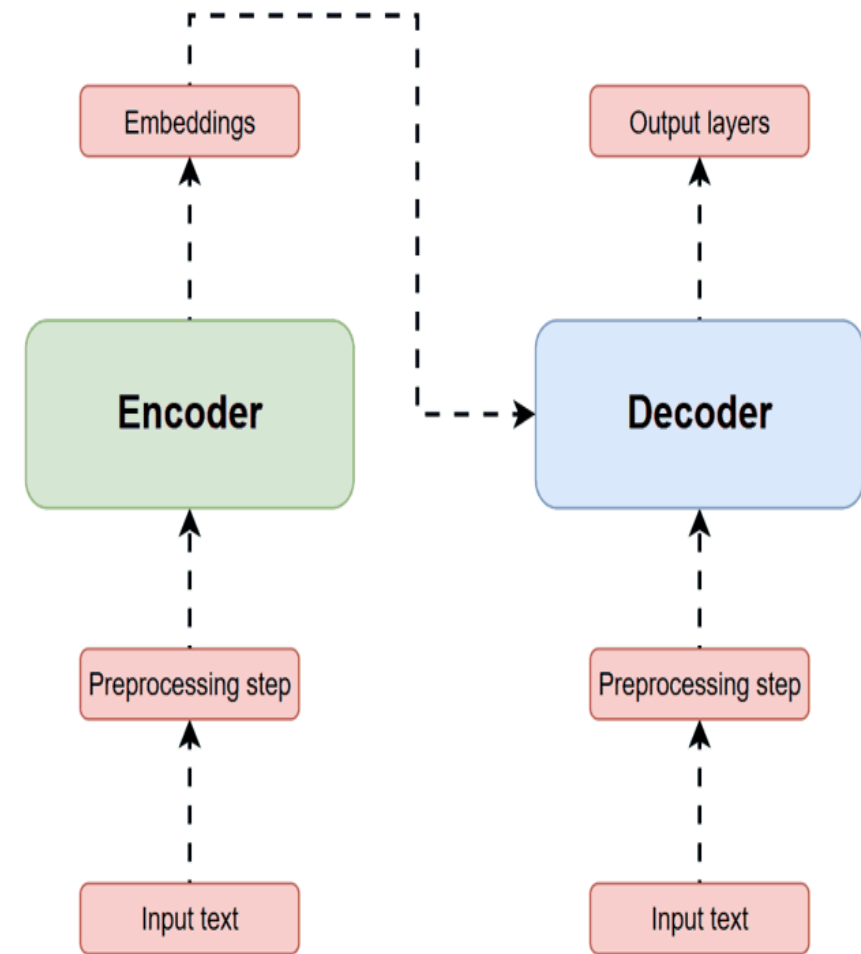
Transformer models are a type of neural network architecture that use a **self-attention mechanism** to perform machine translation and other language tasks. They revolutionized the field by enabling **parallel processing** of entire text sequences, which makes them faster and more efficient than previous models like Recurrent Neural Networks (RNNs)

The Encoder

- Function: Processes the entire input sequence (e.g., the source sentence).
- Output: Generates a set of context-rich representations that capture intricate patterns and relationships within the input data

The Decoder

- Function: Generates the output sequence (e.g., the translated sentence) token by token.
- Mechanism: It uses an attention mechanism to dynamically focus on (attend to) the most relevant parts of the encoder's representations and the tokens it has already produced.



Advantages and Limitation Transformer Models



- **Parallel Processing:** Enables faster training and handling of large datasets by processing tokens simultaneously (unlike RNNs).
- **Long-Range Dependencies:** The self-attention mechanism excels at capturing complex relationships over long sequences
- **State-of-the-Art (SOTA) Performance:** Models like **BERT**, **GPT**, and **T5** (Text-To-Text Transfer Transformer) have achieved new benchmarks in various NLP tasks



Advantages and Limitation Transformer Models



- **Parallel Processing:** Enables faster training and handling of large datasets by processing tokens simultaneously (unlike RNNs).
- **Long-Range Dependencies:** The self-attention mechanism excels at capturing complex relationships over long sequences
- **State-of-the-Art (SOTA) Performance:** Models like BERT, GPT, and T5 (Text-To-Text Transfer Transformer) have achieved new benchmarks in various NLP tasks



- **Computational Resources:** Training large models requires significant resources, such as powerful GPUs or TPUs
- **Complexity:** The architecture is more complex than traditional RNNs, making it harder for beginners to implement and understand





EMERGING &
COLLABORATIVE
STUDIES
